Training a small GPT on TinyStories under a one-hour budget: model size, token exposure, and comparison limits

November 9, 2026

Group Composition:

1. Lara Muy Steffany Mishell (Steffany Lara), A0350565H
2. Kerem Ozemre
3. Kowalke Jeri Tonio Janek (Tonio Kowalke)
4. Haj Hassine Ahmed Ben Abdellaziz (Ahmed Haj Hassine)

Github Repository: github.com/KeremOzemre/Optimization-for-Large-Scale-Data

Abstract

We trained 20 nanoGPT models from random weights on TinyStories, each for one hour on an H100 MIG slice: five sizes (16–253M parameters) crossed with four data fractions (25/50/75/100%). XL (≈124M) has the lowest recorded validation loss among the tested families under this shared recipe. XXL processes about half as many tokens. This is a conditional fixed-time result; reduced exposure is a hypothesis for the ranking, not an established cause. Full-data exposure ranges from 0.303 to 2.077 cache-equivalents, with no guarantee of unique coverage. The original validation-selected `xl_data50` has test loss 1.4846 and perplexity 4.41. Unequal validation samples and one training seed limit close comparisons. Common validation scoring, repeated seeds, longer budgets, and equal-token comparisons are prepared but have no follow-up results yet.

1 Compute budget declaration

Every model starts from random weights (nanoGPT initialisation, with no pretrained weights, embeddings, or tokenizer fine-tuning) and trains for approximately one hour (3600 s) of in-loop wall-clock per model. Data preparation, the final validation pass, and checkpointing all happen outside that timed loop. All 20 runs use the same hardware: a single NVIDIA H100 NVL MIG 3g.47gb slice in `bfloat16`. The same hour of wall-clock does not buy the same number of tokens for every size, and that gap is central to interpreting our result (§5).

2 Research question

Which model size and training-data fraction give the best language-modeling performance under a fixed one-hour budget on identical hardware? We hold the tokenizer, optimizer, schedule, context length, and effective batch size fixed, and vary only (a) model size and (b) the fraction of the training split used.

3 Experimental setup

- **Data:** `roneneldan/TinyStories`. We train on the official train split, and split the official validation set 80/20 into validation and test at the story level (seed 42). All 20 archived test results have been inspected; future model choices use validation (§5.2).
- **Tokenizer:** GPT-2 (`tiktoken`), vocabulary 50,257, with an end-of-text token after every story. Since the tokenizer never changes, per-token cross-entropy and perplexity are directly comparable across all our runs.
- **Model:** nanoGPT [3] GPT with tied input/output embeddings, causal self-attention, context length 256, and dropout 0.1.
- **Evaluation objective:** held-out cross-entropy on the test split, also reported as perplexity ($\exp$ of the loss) and bits per byte. Cross-entropy is the quantity the model is trained on, measured on unseen text, so it reads language-modeling quality directly; perplexity is its interpretable version; and bits per byte lets our numbers be compared to work that uses a different tokenizer [1]. Splitting at the story level with seed 42 keeps any single story from leaking between validation and test.

Table 1: The five model sizes. Width divides evenly by the head count; context length (256) and dropout (0.1) are shared. We adjust microbatch and accumulation to keep an effective batch of 16 sequences (4096 tokens) per optimizer step at every size.

Size	Layers	Width	Heads	Params (M)	Microbatch $\times$ accum.
`small`	4	256	4	16.08	16×1
`medium`	6	384	6	30.02	16×1
`large`	8	512	8	51.04	16×1
`xl`	12	768	12	123.75	8×2
`xxl`	16	1024	16	253.09	4×4

Data fractions: 25/50/75/100% of the training split, giving a 5×4 grid of 20 runs. **Optimizer and schedule:** AdamW (lr 3×10^{-4}, weight decay 0.1, betas 0.9/0.95, gradient clip 1.0), with a time-based 2% warmup and then cosine decay to 10% of the peak learning rate.

4 Methodology and reproducibility

The exact commands are in the repository README; the pipeline is five steps.

1. `run.py prepare` builds (or reuses) the token cache for a given data fraction.
2. Archived `run_config.json` files record the actual settings; checked-in YAMLs provide runnable configurations.
3. `run.py train` trains one config; its outputs (`run_config.json`, `metrics.csv`, `checkpoint.pt`) land in `results/`. Checkpoints and logs are mirrored to the Hugging Face Hub (`dsa4212-tinystories`).
4. The original selection used *validation* loss; all 20 test scores were subsequently exported and inspected.
5. `eval_hf.py` downloads every checkpoint and scores the full test split; `analyze.py` then aggregates all runs into `analysis/` (the tables and figures below).

Validation protocol limitation. Original validation used 16 microbatches: 65,536 targets for Small/Medium/Large, 32,768 for XL, and 16,384 for XXL. Close gaps require common-target scoring. New follow-ups score $4 \times 64 \times 256 = 65{,}536$ targets with a fixed evaluation seed, independent of training randomness.

5 Results

Table 2 lists all 20 runs sorted by final validation loss, the metric we select on. We read three things off these runs, shown in Figures 1–3.

Best observed size under the shared recipe. Validation loss falls approximately $1.84 \to 1.64 \to 1.55 \to 1.50$ from `small` to `xl`. The best recorded XXL loss is 1.5106, compared with 1.5025 for XL. XXL has about twice as many parameters and processes roughly half as many targets. Reduced exposure, optimizer suitability, and training randomness are possible explanations; this experiment does not distinguish them. Related compute-allocation work motivates these hypotheses [2], but our five sizes and one seed do not establish a scaling law or precise optimal size.

Small observed data-fraction effects. Within a family the observed spread is about 0.007–0.014 nats, without a consistent monotonic pattern. The 25% cache has 118,450,614 tokens and the full cache 473,992,236. The earlier claim that no run processed as many tokens as the smallest cache was incorrect: several runs processed multiple cache-equivalents. A redundant or homogeneous corpus could contribute to the weak fraction effect, but its cause and stability across seeds are not established.

Table 2: All 20 runs, sorted by final validation loss (the selection metric). "Tokens" is how many training tokens the run actually processed inside the one-hour budget. Test numbers are shown for completeness, but we did not use them to select the model (§5.2).

Run	Params (M)	Data%	Tokens (M)	Val loss	Val ppl	Test ppl
xl_data50	123.75	50	273.9	**1.5025**	**4.49**	4.41
xl_data100	123.75	100	279.8	1.5068	4.51	4.40
xl_data75	123.75	75	269.1	1.5099	4.53	4.43
xxl_data100	253.09	100	143.7	1.5106	4.53	4.56
xl_data25	123.75	25	283.3	1.5118	4.54	4.45
xxl_data75	253.09	75	141.1	1.5125	4.54	4.59
xxl_data50	253.09	50	148.2	1.5155	4.55	4.56
xxl_data25	253.09	25	143.2	1.5233	4.59	4.61
large_data100	51.04	100	562.4	1.5460	4.69	4.66
large_data75	51.04	75	562.0	1.5466	4.70	4.67
large_data50	51.04	50	557.9	1.5479	4.70	4.69
large_data25	51.04	25	563.2	1.5509	4.72	4.70
medium_data100	30.02	100	734.6	1.6444	5.18	5.16
medium_data75	30.02	75	738.3	1.6462	5.19	5.17
medium_data50	30.02	50	737.3	1.6527	5.22	5.20
medium_data25	30.02	25	719.1	1.6558	5.24	5.21
small_data75	16.08	75	940.8	1.8383	6.29	6.28
small_data100	16.08	100	984.3	1.8386	6.29	6.26
small_data50	16.08	50	985.5	1.8421	6.31	6.29
small_data25	16.08	25	972.0	1.8452	6.33	6.31

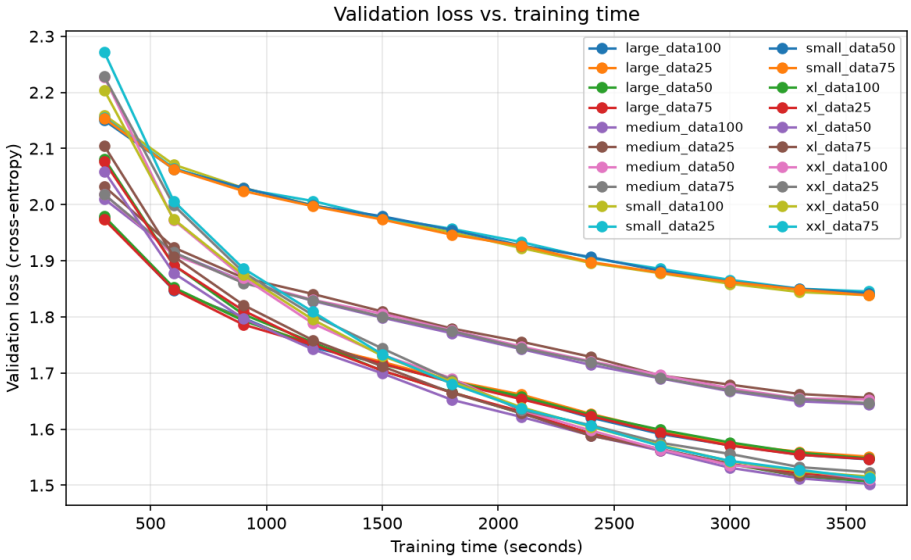


Figure 1: Original validation loss curves. Late improvements do not establish convergence. Validation sample sizes differ by model family.

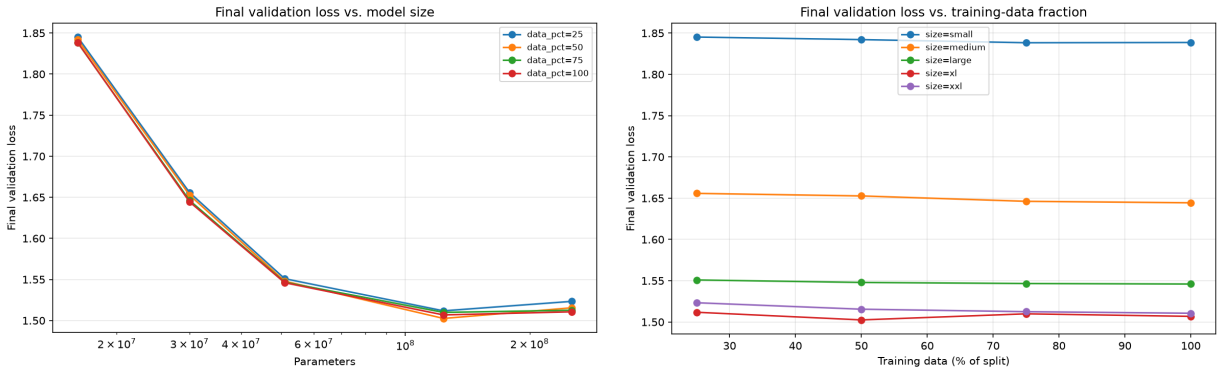


Figure 2: Recorded final validation losses by model size and data fraction. Close differences require common-target scoring and repeated training seeds.

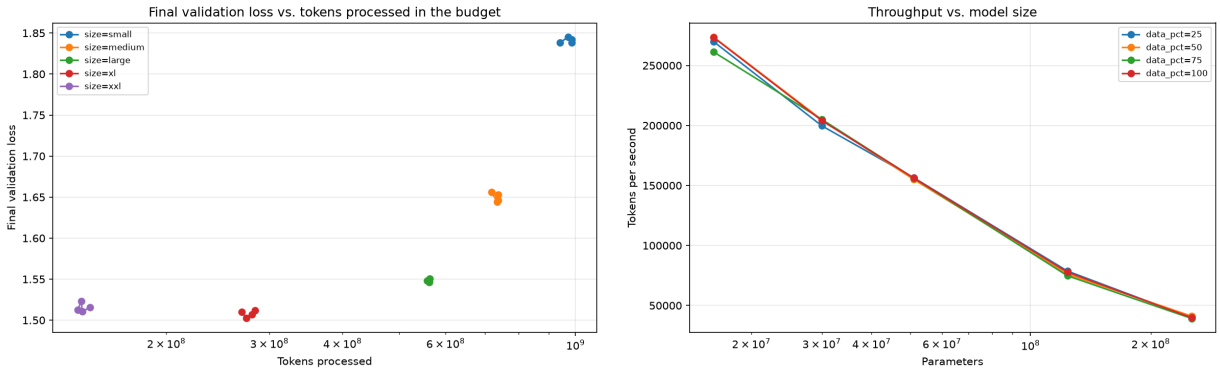


Figure 3: Processed targets and throughput under the original one-hour budget. Larger models process fewer targets; these plots do not isolate the cause of loss differences.

5.1 Token exposure audit

All 262,432 logged counters match completed optimizer steps times microbatch, accumulation, and context length. Table 3 shows full-data runs. Sampling is with replacement: equivalents count repeated targets and do not certify an epoch or unique coverage. Time estimates assume the original measured throughput.

Table 3: Full-cache exposure and estimated time for one equivalent.

Model	Targets (M)	Equivalents	Time for one equivalent
Small	984.326	2.077	28m 54s
Medium	734.642	1.550	38m 43s
Large	562.430	1.187	50m 34s
XL	279.831	0.590	1h 42m
XXL	143.655	0.303	3h 18m

5.2 Archived test results

The original validation selection gives `xl_data50`, with test loss 1.4846, perplexity 4.41, and 0.5237 bits per byte. Full-data XL has test loss 1.4824; full-data XXL has 1.5174. All 20 test scores have subsequently been inspected. They must be disclosed as exploratory comparisons; future adaptive decisions use validation and cannot treat this test set as fresh confirmation. The archived full-test protocol scores 957,184 targets and drops a short tail. The new full evaluator includes the tail and records that protocol change.

We have no training repeats supporting a claim that fraction differences are “within noise” or statistically indistinguishable. Equal tokenization makes CE comparable as an objective, but identical targets and context policy are also needed for a close numerical comparison.

5.3 Qualitative observations

Existing examples show fluent TinyStories-style text even from smaller models. Some XL examples appear more coherent, but the five-prompt, single-sample, unblinded inspection is illustrative rather than a quantified capability result. Frozen prompts, two generation seeds, blind rating sheets, and paired color-tracking probes are prepared. These narrow probes do not establish general reasoning ability.

5.4 Exploratory size and compute plots

The proxy $C \approx 6ND$ uses parameter count N and processed targets D; it is not a hardware measurement. It does not establish GPU utilization. A quadratic fit to five size-level losses is descriptive and depends on the functional form, single seed, evaluation samples, and chosen points. The updated fit uses 100% data for every size; its vertex is not evidence of a compute-optimal parameter count.

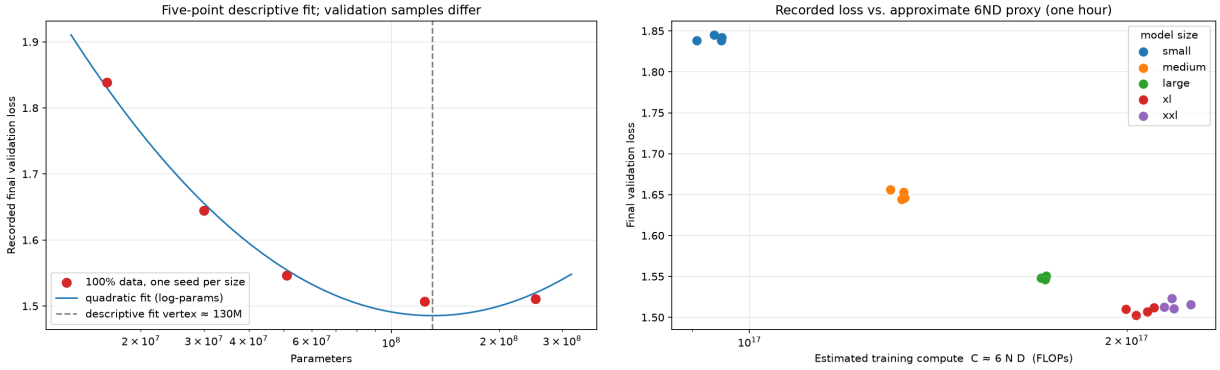


Figure 4: Descriptive size fit on full-data runs and the approximate $6ND$ proxy. Neither panel establishes an optimal size, a scaling law, or measured GPU utilization.

5.5 Additional budget and context runs: exploratory

The 30-minute Large, XL, and XXL runs have recorded validation losses 1.580, 1.633, and 1.680. Large has the lowest recorded value in that group; XL leads at one hour. However, the additional runs record `NVIDIA H100 NVL`, while the main grid records `NVIDIA H100 NVL MIG 3g.47gb`. Validation sample sizes also differ. This does not isolate a causal budget effect. The updated budget plot fixes the data fraction at 100%, but hardware still needs reconciliation.

The XL 512-token-context run records validation loss 1.4555 versus 1.5068 for the 256-token full-data baseline, and processes 190M versus 280M targets. Besides the hardware label, effective targets per optimizer step change from 4096 to 8192 and validation windows change. The observed difference cannot be attributed solely to context.

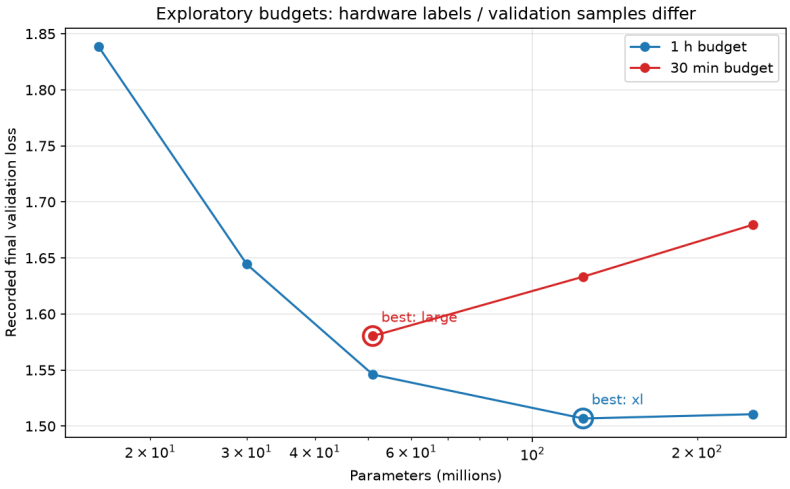


Figure 5: Exploratory recorded losses at two budgets, with 100% data. Different hardware labels and validation samples prevent a controlled budget interpretation.

6 Discussion and limitations

The grid addresses a practical question about a fixed-time training recipe. Equal time intentionally allows different token exposure and does not invalidate that question. It does limit claims about intrinsic capacity, convergence, or performance at equal exposure. Whether XXL overtakes XL with more training remains open.

Only one training seed was used. The optimizer recipe was not tuned per size; validation samples differed by family; windows were sampled with replacement; the dataset revision was not pinned; and all 20 test scores have been inspected. These limitations cannot be resolved from learning curves alone.

6.1 Follow-ups prepared; results pending

Exact commands are in `FOLLOWUP_COMMANDS.md`. None of the following has produced follow-up research results yet:

1. Score archived Large/full, XL/50%, XL/full, and XXL/full checkpoints on identical validation targets. Save paired story-level losses and conditional bootstrap intervals.

2. Train XL and XXL on full data for one hour with seeds 42, 43, and 44 (six model-hours). Report individual scores and training-seed variability.

3. Train a fresh XL/XXL pair for four hours each (eight model-hours). The learning-rate schedule stretches with the budget; these are not checkpoint continuations.

4. Optionally compare both models at 115,721 updates, or 473,993,216 targets, with step-based schedules. A six-hour safety cap marks early stops as incomplete; equal targets still do not

guarantee an epoch.

5. Generate two samples for each of 20 frozen prompts, use two blind raters, and run paired color-tracking probes. Keep these exploratory results separate from CE.

New configs fix preparation seed 42, evaluation seed 43, and 65,536 in-loop validation targets independently of the training seed. Training records code and cache hashes, package versions, actual hardware, targets, and stop reason. Story-resampling intervals condition on fixed checkpoints and do not replace training repeats. Three seeds are an initial variability check, not a precise uncertainty guarantee. Four-hour and equal-token comparisons answer different questions.

7 Conclusion

XL is the best observed family among the tested one-hour configurations under the shared recipe. The original validation-selected model reaches test perplexity 4.41. This supports a bounded fixed-time observation, rather than a universal claim that larger models are better or a precise compute-optimal size. Data-fraction effects are small in the recorded single-seed runs. Their stability, the cause of the XL–XXL difference, and longer-budget or equal-token behavior remain to be tested. Pending experiments are not evidence for the current conclusion.

References

[1] Ronen Eldan and Yuanzhi Li. Tinystories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.

[2] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[3] Andrej Karpathy. nanogpt. https://github.com/karpathy/nanoGPT, 2023. commit 3adf61e.

A Generated story samples

The original five-prompt samples for all 20 runs are in `analysis/generations_all.md`. These examples were inspected without blinding and are illustrative, not a scored capability benchmark. New blind samples and ratings are pending.